

Understanding HTML5 Canvas

>Canvas was one of the foremost elements proposed in HTML5, along with video and audio. Here's a closer look at the Flash-killer feature > by Kshitij Sobti

Canvas is a new HTML element. As the term suggests, it forms a canvas on which a developer can paint anything. It is one of the few—perhaps only—HTML elements that is rendered useless without JavaScript. Using JavaScript, a developer can draw shapes, and render images on the canvas. If JavaScript is disabled in a browser however, the canvas element will show its fallback content i.e. the content inside the tag instead of the tag itself.

Since the canvas gives developers an open area to play with, it can be quite useful to draw custom interfaces such as games. It can also be used to build complex image editors. An image can be rendered on a canvas, modified, and finally passed on to the user who can save it as a file. If that didn't get you excited, a developer can even pull content from a video source, apply transformations, and render it on a canvas to have a video editor or filter of sorts.

We're stressing on the term developer, because the canvas tag is entirely operated via a JavaScript API. This also means that the canvas tag is impenetrable to anyone who doesn't know JavaScript. Without using third-party libraries or tools, a designer cannot create content that will show up on a canvas.

The canvas tag has been around for a while now, it has been supported since Firefox 1.5, Chrome 1.0, Opera 9.0, Safari 2.0, and is even supported in Internet Explorer since 9.0.

The Canvas Tag

It's a simple tag. All it needs to do is display content. So it just defines a width and height, and leaves the rest to the user. Here is how you would define a canvas in an HTML document:

```
<canvas id="mycanvas" width="800" height="600">
```

The text that is displayed if the canvas tag is not supported, or if JavaScript is disabled.

```
</canvas>
```

That's all to it. Even these are optional; by default a canvas has a width of 300 pixels and a height of 150 pixels. Next we move to using this tag in code.

Verifying Canvas Support

Although modern browsers may support it, canvas is still missing from older yet widespread browsers such as IE 7.0 and 8.0. It is wise to first check if the canvas tag is supported.

Let us take the above canvas tag as example and see how we would detect the presence of canvas functionality:

```
var canvas = document.getElementById('mycanvas');
if (canvas.getContext) [
  // Canvas is supported
] else [
  // canvas is not supported
]
```

In our examples, we will leave out this bit of code to save space, but do use it in real applications.

The Canvas Context

You might have noticed the `getContext` bit above, and wondered what it is. Fact is, the canvas has multiple “contexts” which use different rendering systems. The context we will concern ourselves with in this article, is called simply “2d” and is used for 2d rendering. The other context is “webgl” which can be used for 3D hardware accelerated content in the browser; all in the canvas. The WebGL context is not well supported as of now.

We can obtain the “2d” context quite simply as follows:

```
var canvas = document.getElementById('mycanvas');
var canvas2d = canvas.getContext('2d');
```

Replace the ‘2d’ bit in `getContext` with “webgl” and you have the WebGL context, but that is beyond the scope of this article. Now that we have the 2d context, we can begin drawing on it.

Drawing shapes on the canvas

The canvas has a bare minimum API for drawing stuff. In fact, the only primitive you can draw is a rectangle! So let's look at how you would draw that first:

```
var canvas = document.getElementById('mycanvas');
var ctx = canvas.getContext('2d');
ctx.strokeRect(20, 10, 30, 15);
```

This above code will draw a rectangle 20 pixels from the left, 10 pixels from the top, with a width of 30 pixels and a height of 15 pixels. The `strokeRect` method draws an empty rectangular area. If you want a filled rectangle, or if you want to clear out an existing rectangular area, the API is quite similar. Here are these three methods together:

```
strokeRect(x, y, width, height);
fillRect(x, y, width, height);
clearRect(x, y, width, height);
```

For more complex shapes, or even simple shapes such as circles, you have a powerful API for creating any kind of path.